

LeetCode SQL 50 — Complete Master Revision Guide

Purpose: This document is designed as an **all-in-one SQL revision doc**. Read through this guide before any SQL interview or coding exam to refresh **core syntax**, **execution order**, **JOIN patterns**, **window functions**, **date/string manipulation**, and **all 50 LeetCode SQL 50 solutions with concept triggers**.

Table of Contents

1. [SQL Quick-Reference & Fundamentals](#)
 - [1. Order of Execution \(Crucial for Interviews\)](#)
 - [2. Three-Valued Logic & NULL Handling](#)
 - [3. JOIN Cheat Sheet & Visual Guide](#)
 - [4. Essential SQL Functions Toolkit](#)
2. [LeetCode SQL 50 — All Modules & Problems](#)
 - [Module 1: Basic Select \(#1 – #5\)](#)
 - [Module 2: Basic Joins \(#6 – #14\)](#)
 - [Module 3: Basic Aggregate Functions \(#15 – #22\)](#)
 - [Module 4: Sorting and Grouping \(#23 – #29\)](#)
 - [Module 5: Advanced Select and Joins \(#30 – #35\)](#)
 - [Module 6: Subqueries \(#36 – #42\)](#)
 - [Module 7: Advanced String Functions / Regex / Clause \(#43 – #50\)](#)
3. [Top 10 SQL Interview Pitfalls & Quick Checklist](#)

PART 1: SQL Quick-Reference & Fundamentals

1. Order of Execution

Understanding the logical execution order of a SQL query is the #1 way to avoid syntax errors (e.g., using column aliases in `WHERE`).

1. FROM / JOIN → Load & merge base tables
2. WHERE → Filter individual rows (before grouping)
3. GROUP BY → Group rows by key columns
4. HAVING → Filter aggregated groups
5. SELECT → Compute output columns & expressions
6. DISTINCT → Remove duplicate rows
7. ORDER BY → Sort the final result set
8. LIMIT / OFFSET → Return a subset of rows

Important

- **Why you cannot use SELECT aliases in WHERE :** WHERE runs **before** SELECT .
- **WHERE vs HAVING :** Use WHERE for individual row filtering; use HAVING for filtering results of aggregates (COUNT , SUM , AVG).

2. Three-Valued Logic & NULL Handling

In SQL, any comparison with NULL returns UNKNOWN (neither TRUE nor FALSE).

- `col = NULL` ❌ Always evaluates to UNKNOWN .
- `col IS NULL` / `col IS NOT NULL` ✅ Correct way to test for NULL.
- `col != 2` ⚠️ **Will drop rows where col IS NULL !** Always use `col != 2 OR col IS NULL` if you want to include NULLs.

Function	What It Does	Example usage
COALESCE(val1, val2, ...)	Returns the first non-null value	COALESCE(bonus, 0)
IFNULL(val, default)	MySQL specific; replaces NULL with default	IFNULL(price, 0)

Function	What It Does	Example usage
IF(cond, true_val, false_val)	Inline conditional (like ternary ? :)	IF(state='approved', 1, 0)
CASE WHEN ... THEN ... END	Multi-branch conditional logic	CASE WHEN age > 60 THEN 'Senior' ELSE 'Adult' END

3. JOIN Cheat Sheet & Visual Guide

- **INNER JOIN** : Returns rows that have matching values in **both** tables.
- **LEFT JOIN** : Returns **all** rows from the left table, and matched rows from the right table (**NULL** if no match).
- **RIGHT JOIN** : Returns **all** rows from the right table, and matched rows from the left table.
- **CROSS JOIN** : Cartesian product (every row of Table A paired with every row of Table B).
- **Self-Join**: Joining a table to itself (useful for hierarchy, consecutive days, comparing pairs).
- **Anti-Join**: Finding unmatched rows using `LEFT JOIN ... WHERE right_table.id IS NULL` .

4. Essential SQL Functions Toolkit

Date & Time (MySQL)

- DATEDIFF(date1, date2) : Returns difference in days (date1 – date2).
- DATE_ADD(date1, INTERVAL 1 DAY) / DATE_SUB(date1, INTERVAL 1 DAY) : Add or subtract time intervals.
- DATE_FORMAT(date_col, '%Y-%m') : Formats date as string (e.g., '2020-02').
- YEAR(date_col) , MONTH(date_col) : Extracts numeric year or month.

String Manipulation

- CONCAT(str1, str2, ...) : Join strings together.
- SUBSTRING(str, start_idx, length) : Extract substring (**1-indexed** in SQL!).
- UPPER(str) / LOWER(str) : Case conversion.
- CHAR_LENGTH(str) : Number of characters in a string.
- GROUP_CONCAT(col ORDER BY col SEPARATOR ',') : Combines multiple rows of text into a single comma-separated string.

Window Functions

- `ROW_NUMBER() OVER (PARTITION BY col ORDER BY col2)` : 1, 2, 3, 4 (no duplicates).
- `RANK() OVER (PARTITION BY col ORDER BY col2)` : 1, 2, 2, 4 (leaves gaps on ties).
- `DENSE_RANK() OVER (PARTITION BY col ORDER BY col2)` : 1, 2, 2, 3 (no gaps on ties).
- `LEAD(col, 1) OVER (...)` / `LAG(col, 1) OVER (...)` : Look ahead / look behind 1 row.
- `SUM(col) OVER (ORDER BY col ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)` : Rolling 7-day sum.

PART 2: LeetCode SQL 50 — All Modules & Problems

Module 1: Basic Select

1. Recyclable and Low Fat Products (1757)

- **Concept:** Simple `WHERE` clause combining conditions with `AND`.

```
SELECT product_id
FROM Products
WHERE low_fats = 'Y' AND recyclable = 'Y';
```

2. Find Customer Referee (584)

- **Concept:** Three-valued logic. `!= 2` excludes `NULL`, so explicit `IS NULL` check is mandatory.

```
SELECT name
FROM Customer
WHERE referee_id != 2 OR referee_id IS NULL;
```

3. Big Countries (595)

- **Concept:** Filtering with `OR` condition across multiple metrics.

```
SELECT name, population, area
FROM World
WHERE area >= 3000000 OR population >= 25000000;
```

4. Article Views I (1148)

- **Concept:** Self-referencing IDs (`author_id = viewer_id`) + `DISTINCT` to remove duplicate read events.

```
SELECT DISTINCT author_id AS id
FROM Views
WHERE author_id = viewer_id
ORDER BY id ASC;
```

5. Invalid Tweets (1683)

- **Concept:** String length validation. Use `CHAR_LENGTH()` (characters) instead of `LENGTH()` (bytes).

```
SELECT tweet_id
FROM Tweets
WHERE CHAR_LENGTH(content) > 15;
```

Module 2: Basic Joins

6. Replace Employee ID With The Unique Identifier (1378)

- **Concept:** `LEFT JOIN` preserves all employees even if they don't have a matching unique ID in `EmployeeUNI`.

```
SELECT eu.unique_id, e.name
FROM Employees e
LEFT JOIN EmployeeUNI eu ON e.id = eu.id;
```

7. Product Sales Analysis I (1068)

- **Concept:** Standard `INNER JOIN` linking fact table (`Sales`) to dimension table (`Product`).

```
SELECT p.product_name, s.year, s.price
FROM Sales s
JOIN Product p ON s.product_id = p.product_id;
```

8. Customer Who Visited but Did Not Make Any Transactions (1581)

- **Concept:** **Anti-Join pattern** using `LEFT JOIN + WHERE right_id IS NULL` to find visits without transactions.

```
SELECT v.customer_id, COUNT(v.visit_id) AS count_no_trans
FROM Visits v
LEFT JOIN Transactions t ON v.visit_id = t.visit_id
WHERE t.transaction_id IS NULL
GROUP BY v.customer_id;
```

9. Rising Temperature (197)

- **Concept: Self-Join with Date Math.** Must use `DATEDIFF(day1, day2) = 1` to ensure consecutive days.

```
SELECT w1.id
FROM Weather w1
JOIN Weather w2 ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
WHERE w1.temperature > w2.temperature;
```

10. Average Time of Process per Machine (1661)

- **Concept:** Self-Join pairing 'start' activity rows with 'end' activity rows on the same machine/process.

```
SELECT a1.machine_id, ROUND(AVG(a2.timestamp - a1.timestamp), 3) AS processing_time
FROM Activity a1
JOIN Activity a2
  ON a1.machine_id = a2.machine_id
  AND a1.process_id = a2.process_id
  AND a1.activity_type = 'start'
  AND a2.activity_type = 'end'
GROUP BY a1.machine_id;
```

11. Employee Bonus (577)

- **Concept:** `LEFT JOIN` + `NULL` filtering. Employees with no bonus row have `bonus IS NULL`.

```
SELECT e.name, b.bonus
FROM Employee e
LEFT JOIN Bonus b ON e.empId = b.empId
WHERE b.bonus < 1000 OR b.bonus IS NULL;
```

12. Students and Examinations (1280)

- **Concept:** `CROSS JOIN` creates every combination of (Student, Subject), followed by `LEFT JOIN` to actual exams.

```
SELECT s.student_id, s.student_name, sub.subject_name, COUNT(e.subject_name) AS attended_exams
FROM Students s
CROSS JOIN Subjects sub
LEFT JOIN Examinations e
  ON s.student_id = e.student_id
  AND sub.subject_name = e.subject_name
GROUP BY s.student_id, s.student_name, sub.subject_name
ORDER BY s.student_id, sub.subject_name;
```

13. Managers with at Least 5 Direct Reports (570)

- **Concept:** Filtering using `IN` (subquery with `HAVING`) or joining against an aggregated subquery.

```
SELECT name
FROM Employee
WHERE id IN (
    SELECT managerId
    FROM Employee
    GROUP BY managerId
    HAVING COUNT(*) >= 5
);
```

14. Confirmation Rate (1934)

- **Concept: Conditional Aggregation** using `AVG(condition)` which averages `1` for true and `0` for false.

```
SELECT s.user_id, ROUND(COALESCE(AVG(c.action = 'confirmed'), 0), 2) AS confirmation_rate
FROM Signups s
LEFT JOIN Confirmations c ON s.user_id = c.user_id
GROUP BY s.user_id;
```

Module 3: Basic Aggregate Functions

15. Not Boring Movies (620)

- **Concept:** Odd numbers via modulo operator (`id % 2 = 1`) + simple `WHERE` and `ORDER BY`.

```
SELECT *
FROM cinema
WHERE id % 2 = 1 AND description != 'boring'
ORDER BY rating DESC;
```

16. Average Selling Price (1251)

- **Concept:** Weighted average formula: `SUM(price * units) / SUM(units)` with range join on date (`BETWEEN`).

```

SELECT p.product_id, IFNULL(ROUND(SUM(p.price * u.units) / SUM(u.units), 2), 0) AS average_price
FROM Prices p
LEFT JOIN UnitsSold u
    ON p.product_id = u.product_id
    AND u.purchase_date BETWEEN p.start_date AND p.end_date
GROUP BY p.product_id;

```

17. Project Employees I (1075)

- **Concept:** Standard AVG() with ROUND(..., 2) grouped by foreign key.

```

SELECT project_id, ROUND(AVG(experience_years), 2) AS average_years
FROM Project p
JOIN Employee e ON p.employee_id = e.employee_id
GROUP BY project_id;

```

18. Percentage of Users Attended a Contest (1633)

- **Concept:** Dividing group count by scalar subquery (SELECT COUNT(*) FROM Users) for global percentage.

```

SELECT contest_id, ROUND(COUNT(user_id) * 100.0 / (SELECT COUNT(*) FROM Users), 2) AS percentage
FROM Register
GROUP BY contest_id
ORDER BY percentage DESC, contest_id ASC;

```

19. Queries Quality and Percentage (1211)

- **Concept:** Multiple conditional aggregations in a single query (AVG(rating/position) and AVG(rating < 3)*100).

```

SELECT query_name,
    ROUND(AVG(rating / position), 2) AS quality,
    ROUND(AVG(rating < 3) * 100, 2) AS poor_query_percentage
FROM Queries
WHERE query_name IS NOT NULL
GROUP BY query_name;

```

20. Monthly Transactions I (1193)

- **Concept:** Multi-dimensional GROUP BY on formatted month (DATE_FORMAT) and country, using IF() for conditional sums.

```

SELECT DATE_FORMAT(trans_date, '%Y-%m') AS month,
       country,
       COUNT(id) AS trans_count,
       SUM(IF(state = 'approved', 1, 0)) AS approved_count,
       SUM(amount) AS trans_total_amount,
       SUM(IF(state = 'approved', amount, 0)) AS approved_total_amount
FROM Transactions
GROUP BY month, country;

```

21. Immediate Food Delivery II (1174)

- **Concept:** Tuple matching (customer_id, order_date) IN (SELECT customer_id, MIN(order_date) ...) to isolate first orders.

```

SELECT ROUND(AVG(order_date = customer_pref_delivery_date) * 100, 2) AS immediate_percentage
FROM Delivery
WHERE (customer_id, order_date) IN (
    SELECT customer_id, MIN(order_date)
    FROM Delivery
    GROUP BY customer_id
);

```

22. Game Play Analysis IV (550)

- **Concept:** Finding consecutive login on first_login + 1 day by joining on subquery with MIN(event_date) .

```

SELECT ROUND(COUNT(t2.player_id) / COUNT(t1.player_id), 2) AS fraction
FROM (
    SELECT player_id, MIN(event_date) AS first_login
    FROM Activity
    GROUP BY player_id
) t1
LEFT JOIN Activity t2
ON t1.player_id = t2.player_id
AND t2.event_date = DATE_ADD(t1.first_login, INTERVAL 1 DAY);

```

Module 4: Sorting and Grouping

23. Number of Unique Subjects Taught by Each Teacher (2356)

- **Concept:** COUNT(DISTINCT column) to count unique occurrences per group.

```
SELECT teacher_id, COUNT(DISTINCT subject_id) AS cnt
FROM Teacher
GROUP BY teacher_id;
```

24. User Activity for the Past 30 Days I (1141)

- **Concept:** Inclusive date window filtering using BETWEEN '2019-06-28' AND '2019-07-27' .

```
SELECT activity_date AS day, COUNT(DISTINCT user_id) AS active_users
FROM Activity
WHERE activity_date BETWEEN '2019-06-28' AND '2019-07-27'
GROUP BY activity_date;
```

25. Product Sales Analysis III (1070)

- **Concept:** Matching multiple columns (product_id, year) against a MIN(year) grouped subquery.

```
SELECT product_id, year AS first_year, quantity, price
FROM Sales
WHERE (product_id, year) IN (
    SELECT product_id, MIN(year)
    FROM Sales
    GROUP BY product_id
);
```

26. Classes More Than 5 Students (596)

- **Concept:** Using HAVING COUNT(student) >= 5 to filter aggregated groups.

```
SELECT class
FROM Courses
GROUP BY class
HAVING COUNT(student) >= 5;
```

27. Find Followers Count (1729)

- **Concept:** Clean grouping and ascending sorting by primary ID.

```
SELECT user_id, COUNT(follower_id) AS followers_count
FROM Followers
GROUP BY user_id
ORDER BY user_id;
```

28. Biggest Single Number (619)

- **Concept:** Outer `SELECT MAX(num)` wrapper around a subquery to ensure `NULL` is returned if no number appears once.

```
SELECT MAX(num) AS num
FROM (
    SELECT num
    FROM MyNumbers
    GROUP BY num
    HAVING COUNT(num) = 1
) AS single_nums;
```

29. Customers Who Bought All Products (1045)

- **Concept: Relational Division.** Comparing customer's distinct product count against total distinct products in `Product`.

```
SELECT customer_id
FROM Customer
GROUP BY customer_id
HAVING COUNT(DISTINCT product_key) = (SELECT COUNT(*) FROM Product);
```

Module 5: Advanced Select and Joins

30. The Number of Employees Which Report to Each Employee (1731)

- **Concept:** Hierarchy Self-Join linking manager (`mgr.employee_id`) to reportee (`emp.reports_to`).

```
SELECT mgr.employee_id, mgr.name,
       COUNT(emp.employee_id) AS reports_count,
       ROUND(AVG(emp.age)) AS average_age
FROM Employees mgr
JOIN Employees emp ON mgr.employee_id = emp.reports_to
GROUP BY mgr.employee_id, mgr.name
ORDER BY mgr.employee_id;
```

31. Triangle Judgement (610)

- **Concept:** Triangle inequality theorem inside `IF(x+y>z AND x+z>y AND y+z>x, 'Yes', 'No')`.

```
SELECT x, y, z,
       IF(x + y > z AND x + z > y AND y + z > x, 'Yes', 'No') AS triangle
FROM Triangle;
```

32. Consecutive Numbers (180)

- **Concept:** Checking 3 consecutive identical values using `LEAD(col, 1)` and `LEAD(col, 2)`.

```
SELECT DISTINCT num AS ConsecutiveNums
FROM (
    SELECT num,
           LEAD(num, 1) OVER() AS num1,
           LEAD(num, 2) OVER() AS num2
    FROM Logs
) t
WHERE num = num1 AND num = num2;
```

33. Product Price at Given Date (1164)

- **Concept:** Combining latest price before date (`MAX(change_date) <= '2019-08-16'`) with a fallback `UNION` for products that never changed before that date.

```
SELECT product_id, new_price AS price
FROM Products
WHERE (product_id, change_date) IN (
    SELECT product_id, MAX(change_date)
    FROM Products
    WHERE change_date <= '2019-08-16'
    GROUP BY product_id
)
UNION
SELECT DISTINCT product_id, 10 AS price
FROM Products
WHERE product_id NOT IN (
    SELECT product_id
    FROM Products
    WHERE change_date <= '2019-08-16'
);
```

34. Last Person to Fit in the Bus (1204)

- **Concept: Cumulative Sum Window Function** `SUM(weight) OVER (ORDER BY turn)` filtered by `<= 1000`.

```

SELECT person_name
FROM (
    SELECT person_name, SUM(weight) OVER (ORDER BY turn) AS running_weight
    FROM Queue
) t
WHERE running_weight <= 1000
ORDER BY running_weight DESC
LIMIT 1;

```

35. Count Salary Categories (1907)

- **Concept:** Combining 3 independent subqueries via `UNION` so 0-count categories are explicitly included.

```

SELECT 'Low Salary' AS category, SUM(salary < 20000) AS accounts_count FROM Accounts
UNION
SELECT 'Average Salary', SUM(salary >= 20000 AND salary <= 50000) FROM Accounts
UNION
SELECT 'High Salary', SUM(salary > 50000) FROM Accounts;

```

Module 6: Subqueries

36. Employees Whose Manager Left the Company (1978)

- **Concept:** `NOT IN` subquery checking active employee IDs for managers that no longer exist.

```

SELECT employee_id
FROM Employees
WHERE salary < 30000
    AND manager_id NOT IN (SELECT employee_id FROM Employees)
ORDER BY employee_id;

```

37. Exchange Seats (626)

- **Concept:** Mathematical seat-swapping inside a `CASE` expression (odd IDs +1, even IDs -1, last odd ID stays same).

```

SELECT
  CASE
    WHEN id % 2 != 0 AND id = (SELECT MAX(id) FROM Seat) THEN id
    WHEN id % 2 != 0 THEN id + 1
    ELSE id - 1
  END AS id,
  student
FROM Seat
ORDER BY id;

```

38. Movie Rating (1341)

- **Concept:** UNION ALL combining top user by rating count and top movie by average rating in February 2020.

```

(SELECT u.name AS results
FROM MovieRating mr JOIN Users u ON mr.user_id = u.user_id
GROUP BY u.user_id
ORDER BY COUNT(*) DESC, u.name ASC LIMIT 1)
UNION ALL
(SELECT m.title
FROM MovieRating mr JOIN Movies m ON mr.movie_id = m.movie_id
WHERE DATE_FORMAT(mr.created_at, '%Y-%m') = '2020-02'
GROUP BY m.movie_id
ORDER BY AVG(mr.rating) DESC, m.title ASC LIMIT 1);

```

39. Restaurant Growth (1321)

- **Concept: 7-Day Rolling Window** using ROWS BETWEEN 6 PRECEDING AND CURRENT ROW .

```

SELECT visited_on, amount, ROUND(amount/7, 2) AS average_amount
FROM (
  SELECT visited_on,
    SUM(SUM(amount)) OVER (ORDER BY visited_on ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
    COUNT(visited_on) OVER (ORDER BY visited_on ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) A
  FROM Customer
  GROUP BY visited_on
) t
WHERE count_days = 7;

```

40. Friend Requests II: Who Has the Most Friends (602)

- **Concept:** Unpivoting two columns (requester_id and acceptor_id) into a single list using UNION ALL .

```

SELECT id, COUNT(*) AS num
FROM (
    SELECT requester_id AS id FROM RequestAccepted
    UNION ALL
    SELECT acceptor_id AS id FROM RequestAccepted
) t
GROUP BY id
ORDER BY num DESC
LIMIT 1;

```

41. Investments in 2016 (585)

- **Concept:** Multi-column window counts (COUNT(*) OVER(PARTITION BY tiv_2015) and COUNT(*) OVER(PARTITION BY lat, lon)) to check uniqueness conditions.

```

SELECT ROUND(SUM(tiv_2016), 2) AS tiv_2016
FROM (
    SELECT tiv_2016,
           COUNT(*) OVER(PARTITION BY tiv_2015) AS tiv_2015_cnt,
           COUNT(*) OVER(PARTITION BY lat, lon) AS loc_cnt
    FROM Insurance
) t
WHERE tiv_2015_cnt > 1 AND loc_cnt = 1;

```

42. Department Top Three Salaries (185)

- **Concept:** DENSE_RANK() partitioned by department. Ensures ties share ranks and next salary rank is continuous (rnk <= 3).

```

SELECT Department, Employee, Salary
FROM (
    SELECT d.name AS Department, e.name AS Employee, e.salary AS Salary,
           DENSE_RANK() OVER (PARTITION BY d.id ORDER BY e.salary DESC) AS rnk
    FROM Employee e
    JOIN Department d ON e.departmentId = d.id
) t
WHERE rnk <= 3;

```

Module 7: Advanced String Functions / Regex / Clause

43. Fix Names in a Table (1667)

- **Concept:** Capitalizing first letter (UPPER(SUBSTRING(name, 1, 1))) and lowercasing the rest (LOWER(SUBSTRING(name, 2))).

```
SELECT user_id,  
       CONCAT(UPPER(SUBSTRING(name, 1, 1)), LOWER(SUBSTRING(name, 2))) AS name  
FROM Users  
ORDER BY user_id;
```

44. Patients With a Condition (1527)

- **Concept:** LIKE prefix matching for words at the beginning ('DIAB1%') or after a space ('% DIAB1%').

```
SELECT patient_id, patient_name, conditions  
FROM Patients  
WHERE conditions LIKE 'DIAB1%' OR conditions LIKE '% DIAB1%';
```

45. Delete Duplicate Emails (196)

- **Concept:** Self-referencing table delete (DELETE p1 FROM Person p1, Person p2) keeping lowest ID.

```
DELETE p1  
FROM Person p1, Person p2  
WHERE p1.email = p2.email AND p1.id > p2.id;
```

46. Second Highest Salary (176)

- **Concept:** LIMIT 1 OFFSET 1 wrapped in an outer SELECT to convert an empty result into NULL .

```
SELECT (  
    SELECT DISTINCT salary  
    FROM Employee  
    ORDER BY salary DESC  
    LIMIT 1 OFFSET 1  
) AS SecondHighestSalary;
```

47. Group Sold Products By The Date (1484)

- **Concept:** String aggregation using GROUP_CONCAT(DISTINCT col ORDER BY col SEPARATOR ',') .

```
SELECT sell_date,
       COUNT(DISTINCT product) AS num_sold,
       GROUP_CONCAT(DISTINCT product ORDER BY product ASC SEPARATOR ',') AS products
FROM Activities
GROUP BY sell_date
ORDER BY sell_date;
```

48. List the Products Ordered in a Period (1327)

- **Concept:** Filtering order dates in February 2020 (`DATE_FORMAT = '2020-02'`) + `HAVING SUM(unit) >= 100` .

```
SELECT p.product_name, SUM(o.unit) AS unit
FROM Products p
JOIN Orders o ON p.product_id = o.product_id
WHERE DATE_FORMAT(o.order_date, '%Y-%m') = '2020-02'
GROUP BY p.product_id
HAVING SUM(o.unit) >= 100;
```

49. Find Users With Valid E-Mails (1517)

- **Concept:** Regular expressions (`REGEXP`) matching domain, prefix rules, and escaped literal dot (`[.]`).

```
SELECT *
FROM Users
WHERE mail REGEXP '^[a-zA-Z][a-zA-Z0-9_-.]*@leetcode[.]com$';
```

50. Capital Gain/Loss (1393) (Bonus / Core 50th Problem Pattern)

- **Concept:** Using conditional sum inside `SUM()` to treat 'Sell' as positive and 'Buy' as negative cash flow.

```
SELECT stock_name,
       SUM(CASE WHEN operation = 'Sell' THEN price ELSE -price END) AS capital_gain_loss
FROM Stocks
GROUP BY stock_name;
```

PART 3: Top 10 SQL Interview Pitfalls & Quick Checklist

Before submitting any SQL query in an interview, scan this 60-second checklist:

- ☐ **Did you check for NULL values?** Remember that `col != 'value'` excludes NULL rows. Add `OR col IS NULL` if needed.

- ☐ **Did you use WHERE vs HAVING correctly?** WHERE filters individual rows *before* grouping; HAVING filters *after* grouping.
- ☐ **Did you avoid integer division?** In some dialects, `COUNT(x) / COUNT(y)` rounds down to zero. Multiply by `100.0` or cast to float.
- ☐ **Did you handle zero counts when calculating averages?** Wrap aggregations in `COALESCE(AVG(...), 0)` or use `LEFT JOIN` so non-matching rows aren't dropped.
- ☐ **Did you use DENSE_RANK() vs RANK() correctly?** Use `DENSE_RANK()` when finding the Nth highest value so ties don't skip numbers.
- ☐ **Did you use COUNT(DISTINCT col) when duplicates are possible?** Look for wording like "unique users" or "distinct products".
- ☐ **Did you verify inclusive vs. exclusive dates?** When checking ranges, use `BETWEEN` or explicit `>=` and `<=`.
- ☐ **Did you order subquery columns consistently in tuples?** When using `WHERE (col1, col2) IN (SELECT col1, col2 ...)`, the column order in the subquery must match exactly.
- ☐ **Did you wrap fallback queries in (SELECT ...) ?** When an interview asks to "return NULL if no such row exists" (like Second Highest Salary), wrapping the subquery in `SELECT (...) AS col` handles empty sets automatically.
- ☐ **Did you use UNION ALL instead of UNION when duplicates are desired?** `UNION ALL` is significantly faster because it avoids sorting and deduplication.